

look the same, the first link is for creating an issue in the security repo, the second should be for the project (CustomersDot/GitLab) accordingly.

Support Requests

Fulfillment has a new collaborative process with the Support team to track and manage requests for help. We have a specialized, scoped label, ~"Help group::Utilization". Issues are generated in the same project.

Other boards where previous requests are tracked:

- Board: Bugs by severity
- Board: Features by severity

Iteration

Iteration powers Efficiency and is the key that unlocks Results, but it's also really difficult to internalize and practice. Following the development department's key metrics we strive to deliver small and focused MRs.

Exemplary Iteration Attempts

- Epic GitLab.com Billable Members List
 - <<https://gitlab.com/gitlab-org/gitlab/-/issues/321560>>
 - <<https://gitlab.com/gitlab-org/gitlab/-/issues/324658>>
 - <<https://gitlab.com/gitlab-org/gitlab/-/issues/325412>>
 - <<https://gitlab.com/gitlab-org/gitlab/-/issues/321560note543385756>>
- Epic Expired SaaS subscriptions should revert to free plan
- Expired subscriptions

Iteration Retrospectives

Following a similar process to Milestone Retrospectives, we employ Iteration Retrospectives on a quarterly basis.

May 2021

May 2021

Key Takeaways

- Look for blockers as natural boundaries for issue/epic breakdown
- Intentionally cut scope to ensure deliverability. Try to cut scope as early as possible.
- Lean into using the Refinement Template for estimations.
- Remember to find reviewers/maintainers with domain knowledge and compatible timezones for maximum efficiency - See also MR review guidelines
- Share your proof of concept solutions with others to get feedback early on solutions
- Consider acceptable partial solutions - cover the majority case; defer the edge cases for the next iteration if possible

Architecture Reviews

Each month, we conduct an architecture review.

The review process targets reviews from both a current and future design perspective. That is, the group decides if the component under review should be done in a particular way. This is not necessarily "should a thing be done," but it could be used that way. It can also be used to propose a large architectural change that might have wide sweeping changes or as a prelude to the construction of a blueprint in our evolution workflow process.

The timeline has both async and sync steps. For smaller reviews, it may be possible to complete all of the steps asynchronously. We've already established that synchronous time is valuable. This process follows a similar pattern that we have used for our retrospectives and working groups.

Timeline

1. The team creates a review issue
 1. We can use the Utilization team project, the following issue template, and Epic
 1. Find a Reviewer (Issue author and DRI)
 1. Optionally, find a person to chair the review. They would facilitate the process (review meeting scheduling; note taking; managing the clock and discussion).
For smaller reviews, the reviewer also acts as the chair.
 1. Summarize the [concern, feature, proposal, etc]. Limit this to 1 paragraph, but address the problem in its entirety.
The topic should be agreed upon between the team and the reviewer.
 1. The team asynchronously contributes to the issue to provide additional detail that documents the [concern, feature, proposal, etc] identified in the issue description.
 1. Summarize the problem space. Definitions, terms, what is the key motivation for addressing this topic.
 1. List those directly affected by the current problem or future revision. Bring in a stable counterpart (e.g. Support, Sales, cross-stage DRI) to further understand the impact.
 1. Set exit criteria - what we will achieve on completion. Produce a brief that describes when this issue can be closed (e.g. improved documentation, sharing session, epic for refactoring, new issues, a proposal, an architecture blueprint)
 1. Include:
 1. Any architecture being proposed
 1. Non-functional requirements (e.g. accessibility, security, performance, etc)
 1. Relevant business goals
 1. Consider alternatives, costs, and opportunities
 1. Be sure to address any concerns from affected groups or cross-stage collaborators
1. The Reviewer creates a list of questions making them available to the team prior to the review meeting. The questions could be derived from information gathered in the previous step
1. The Reviewer (or Chair) convenes a review meeting. The discussion should be recorded and made as widely available as possible while keeping sensitive information SAFE
 1. Invite the team and any outside collaborator
 1. Invite the EM and SEM/Director

1. PM, Designer, or other internal collaborators can be added optionally depending on the topic

1. Suggested agenda

1. Overview

1. Discussion of questions from step 3 above

1. Is the proposal relevant in a wider context (team goals, sub-department goals, department goals)

1. Action items, if any

1. The Reviewer is responsible for outlining the action items and ultimately deciding the outcome of the review

1. Additional review steps can be completed asynchronously to complete the exit criteria

1. It is possible to alter the questions or direction given discussion. This could include a subsequent review or invalidating the current review.

1. The artifacts of the review (recording, issues, etc) are published and made available to others in the /doc directory of the CustomersDot project

1. Consider updating this timeline with any new learnings or propose adjustments to the issue template if applicable.

1. The review issue is closed

Investment Time

The time required to prepare, discuss, and complete a review is variable.

We should not spend all of our time on the review. We should approach the review pragmatically with a focus on understanding the problem, the value proposition, and definition of success. By meeting on a 30 day cycle, we can spread out the investment required over time.

We should time box the preparation of the review, and we should reconsider or reduce the review scope if the topic is too broad, has too many hidden dependencies, or becomes unwieldy in some other way.

Team collaboration is important to the success of the review. The Reviewer should ask for input, feedback, and receive help from other team members by opening a discussion thread in the review issue. The role is not meant to be solitary, but a typical DRI.

Performance Indicators

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
```

```
  {{< tableau/filters "GROUPLABEL"="utilization" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="utilization" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-
```

```
west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}  
  {{< tableau/filters "GROUPNAME"="utilization" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-  
2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard"  
>}}  
  {{< tableau/filters "GROUPLABEL"="utilization" >}}  
{{< /tableau >}}
```

You can find more centralized engineering metrics at this page.

SECTION: engineering/development/growth/_index.md

Vision

Grow an agile fullstack development capability to support our product goals through feature development and experiments enabling GitLab to make informed product decisions and achieve results in line with our product direction.

Mission

Working inline with our values, we focus on iteration and collaboration, working with and across areas of the product our development department counterparts maintain, as well our own areas of responsibility.

Direction

We work on the issues prioritized by our product teams including running experiments on GitLab.com.

More information on priorities can be found on the Growth direction page.

Growth stage teams have Fullstack Engineers.

The reason for this is that the Growth stage has a need for both Frontend and Backend skill-sets,

but as a small team, has optimized for team member efficiency to adopt the Fullstack role.

Some useful links to see how and what we are working on include:

- Growth direction
- Growth Kanban board for development
- Experimentation
- GLEX
- Experiment rollout
- acquisition group
- activation group

Growth teams contribute to a GitLab experimentation gem to make it easier to run experiments and make data driven product decisions on GitLab.com.

Roadmap

Our current roadmap can be accessed through our quarterly OKR planning issues:

- FY24 Q3
- FY24 Q4

Who Are We?

People Leaders in Development

```
{{< members-by-role role="Growth and Govern" >}}
```

All Team Members

The following people are permanent members of groups that belong to the Growth stage:

Acquisition

```
{{< team-by-departments department="Acquisition Team" >}}
```

Activation

```
{{< team-by-departments department="Activation Team" >}}
```

Counterparts

We work closely with our Product Team counterparts to design and implement features and experiments that measure the impact of changes to our messaging, UX, and overall experience of using GitLab.

Stable Counterparts

The following members of other functional teams are our stable counterparts:

```
{{< engineering/stable-counterparts role="Growth" manager-role="Director of Engineering for Secure, Govern, and Growth" other-manager-roles="Govern and Growth|Growth and Govern:Threat Insights|VP, Online Sales and Self Service" >}}
```

How We Work

Our team follows the Product Development Flow utilizing all labels from ~workflow::start to ~workflow::complete.

We adhere to the Completion Criteria and Who Transitions Out outlined in the Product Development Flow to progress issues from one stage to the next.

Labels and Workflow Boards

We use workflow boards to track issue progress throughout a milestone. Workflow boards should be viewed at the highest group level for visibility into all nested projects in a group.

The Growth stage uses the ~"devops::growth" label and the following groups for tracking merge request rate and ownership of issues and merge requests.

Name	Label	gitlab-org	All Groups
-----	-----	-----	-----
Growth	~"devops::growth"	Growth Workflow	-
Acquisition	~"group::acquisition"	Acquisition Workflow	-
Activation	~"group::activation"	Activation Workflow	-
Experiments	~"experiment-rollout"	Experiment tracking	-
Feature Flags	~"feature flag"	Feature flags	

Growth teams work across the GitLab codebase on multiple groups and projects including:

- The gitlab.com/gitlab-org group
- gitlab
- GLEX
- customers-gitlab-com
- The gitlab.com/gitlab-com group
- www-gitlab-com

Issue sequencing

In order to convey Issue implementation order and blocking concepts, we leverage the blocking issue linking feature.

More on the discussion can be seen in <https://gitlab.com/gitlab-org/growth/team-tasks/-/issues/752>.

Issue Refinement And Estimation

Before the work can begin on an issue, we should refine and estimate it. We have a continuous process for this, leveraging additional workflow status, ~"workflow::refinement", which indicates that the issue is being refined by the team. Once the issue refinement is completed, it can be moved to ~"workflow::scheduling" stage.

The refinement process is driven by triage bot automations and policies to ensure that it's smooth and consistent.

Refinement Steps

1. Issues are moved from ~"workflow::planning breakdown" to ~"workflow::refinement" automatically by the triage bot in order of priority (from top to bottom). The bot will only move issues to refinement if there is room in refinement column, meaning there is less issues than maximum limit for this column. This is first chance for PMs to prioritize issues by moving them higher in the planning breakdown column. After the issue is moved to refinement, a

dedicated refinement thread is created, which acts as a place for discussion and weight estimation.

- · Hint: In rare case when an issue has to be expedited, it's possible to move it to refinement manually. This will invoke a reaction from triage bot, which will add refinement thread for such issue instantly so the refinement can proceed the same way as with automated path.

2. During refinement the team ensures that the issue is well described and requirements are clear. They can use the refinement thread to discuss but they should make sure that any changes and decisions made there are also reflected in issue's description. Once each engineer is comfortable with the way the issue is described, they can vote their estimation of weight based on our guidelines. The voting happens by reacting to the thread with one of few possible weight estimates: 1· 2· 3· 5· or ·.

3. Each day the triage bot checks all issues in ~"workflow::refinement" column and if an issue has required minimum number of estimation votes (see MINREACTIONS constant here for the current setting) it will be moved to ~"workflow::scheduling".

- · Hint: If there is some problem with the issue and it shouldn't be moved forward even if enough engineers estimate it, · reaction can be added to the thread which will stop the bot from transitioning the issue to ~"workflow::scheduling" as long as this reaction sticks to the thread. This means that whoever put it is also responsible for removing it once the problem is gone.

4. Once the issue is in ~"workflow::scheduling", it is awaiting final prioritization by PMs - it has to be manually moved to ~"workflow::ready for dev" depending on the current priorities. This part of the process is PMs responsibility. This allows for additional fine-tuning of priorities and acts as a buffer for our ready for development column.

Estimation guidelines

| Weight | Description (Engineering) |

| ----- | ----- |

| 1 | The simplest possible change. We are confident there will be no side effects. |

| 2 | A simple change (minimal code changes), where we understand all of the requirements. |

| 3 | A simple change, but the code footprint is bigger (e.g. lots of different files, or tests effected). The requirements are clear. |

| 5 | A more complex change that will impact multiple areas of the codebase, there may also be some refactoring involved. Requirements are understood but you feel there are likely to be some gaps along the way. |

| 8 | A complex change, that will involve much of the codebase or will require lots of input from others to determine the requirements. |

| 13 | A significant change that may have dependencies (other teams or third-parties) and we likely still don't understand all of the requirements. It's unlikely we would commit to this in a milestone, and the preference would be to further clarify requirements and/or break in to smaller Issues. |

In planning and estimation, we value velocity over predictability. The main goal of our planning and estimation is to focus on the MVC, uncover blind spots, and help us achieve a baseline level of predictability without over optimizing. We aim for 70% predictability instead of 90%. We believe that optimizing for velocity (merge request rate) enables our Growth teams to achieve a weekly experimentation cadence.

- If an issue has many unknowns where it's unclear if it's a 1 or a 5, we will be cautious and

estimate high (5).

- If an issue has many unknowns, we can break it into two issues. The first issue is for research, also referred to as a Spike), where we de-risk the unknowns and explore potential solutions. The second issue is for the implementation.

- If an initial estimate is incorrect and needs to be adjusted, we revise the estimate immediately and inform the Product Manager. The Product Manager and team will decide if a milestone commitment needs to be adjusted.

Engineering DRI for Large-Scale Initiatives

For substantial epics or projects that require extensive coordination, we designate an Engineering Directly Responsible Individual (DRI). This role is pivotal in managing complex workstreams within our team.

The Engineering <abbr title="Directly Responsible Individual">DRI</abbr> is typically assigned in two scenarios:

1. When an epic reaches the development phase and requires detailed oversight.
1. When an issue grows in scope and complexity, necessitating its promotion to an epic.

Importantly, we appoint the <abbr title="Directly Responsible Individual">DRI</abbr> before breaking down the work into smaller, manageable tasks. This early assignment is crucial because one of the <abbr title="Directly Responsible Individual">DRI</abbr>'s primary responsibilities is to lead the planning process and facilitate the strategic breakdown of the epic or project into actionable deliverables.

By having a dedicated <abbr title="Directly Responsible Individual">DRI</abbr> for these large-scale initiatives, we ensure focused leadership, clear accountability, and effective coordination throughout the development lifecycle of complex engineering efforts.

Role Responsibilities

Communicating Progress and Managing Health Status

The Engineering <abbr title="Directly Responsible Individual">DRI</abbr> oversees weekly updates on the epic, tracking overall health status, including progress, obstacles, and emerging risks.

- They continuously monitor and report on the epic's health status, indicating if it's on track, at risk, or blocked, providing context around any changes.
- Progress updates are reported via a top-level comment on the epic containing the development work (usually linked to our OKRs).
- Updates are due by End of Day on the <abbr title="Directly Responsible Individual">DRI</abbr>'s last workday of the week.

Facilitating Breakdown of Work

This role is crucial in breaking down epics into smaller, manageable deliverables, ensuring tasks are clearly defined and ready for development.

Ensuring Steady Progress

The responsible individual ensures work items progress smoothly through our Kanban flow, with enough work prepared for development to maintain consistent progress.

Risk Management and Mitigation

Early identification of potential risks and proactive collaboration with the team to develop mitigation strategies are key responsibilities. This enhances the role in not only communicating issues but actively managing and resolving them.

Retrospective and Continuous Improvement

Upon conclusion of each epic, a brief retrospective session should be facilitated to gather feedback on successes and areas for improvement. This practice helps refine processes and improves the handling of future epics.

Collaboration with Product and Design

Active collaboration with Product Managers and Designers during the planning phase is essential to ensure the epic's scope aligns with business goals and user experience standards.

Resource Allocation and Escalation

The appointed individual is empowered to identify when additional resources or support are needed and to escalate issues promptly. This helps ensure timelines are met and work progresses smoothly.

Note: While responsible for timely delivery, the <abbr title="Directly Responsible Individual">DRI</abbr> is not expected to execute every task personally.

Standardized Status Update Template

To ensure consistency and clarity in our communication, we've implemented a standardized comment template for status updates across the Growth team.

Template Details:

- Name: "Growth - DRI Status Update"
- Scope: Available for the entire GitLab group
- Location: GitLab.com Comment Templates

This template streamlines our reporting process, making it easier for <abbr title="Directly Responsible Individual">DRI</abbr>s to provide comprehensive and uniform updates. By using a consistent format, we enhance readability and facilitate quick information retrieval for all team members.

For guidance on using comment templates, please refer to our [Comment Templates Usage Guide](#).

Technical exploration ("Spike") guidelines

Occasionally, the team will be asked to dedicate time to exploring or "spiking" a technical solution to a problem or opportunity area deemed a priority by Growth PM. Technical spikes are fundamentally different than our typical work items as the result is more commonly a recommendation on a technical direction or solution rather than code, yet they still require dedicated developer focus and should be accounted for in our development workflows. As such, we've determined the following guidelines and responsibilities for individual Growth Engineers to follow when addressing a technical spike in our workflow:

Note: Technical spikes will serve as the single source of truth for implementation and will have one of three outcomes:
conversion into a development implementation issue, promotion to an epic with additional implementation issues, or
closure if determined not technically feasible.

mermaid

flowchart TD

```
A[Technical Spike Created] --> B[Engineer Assigns Self]
B --> C[Set Due Date with PM]
C --> D[Review & Clarify Requirements]
D --> E[Document Investigation Approach]
E --> F[Investigation Period]
F --> G[Document Progress & Findings]
G --> H[Provide Summary & Recommendations]
H --> I{Technical Feasibility?}
I -->|Not Feasible| J[Close Issue with Explanation]
I -->|Feasible| K{Scope Size?}
K -->|Small Scope| L[Convert to Implementation Issue]
K -->|Large Scope| M[Promote to Epic & Create Implementation Issues]
L --> N[Move to workflow::solution validation]
M --> O[Epic enters workflow process]
```

Inputs

As a technical spike is prioritized and available for pickup in ~workflow::ready for development, complete the following steps:

- Assign yourself, apply ~spike label, set the milestone and set the workflow label to ~workflow::in dev
- Connect with the responsible PM to align on a reasonable due date for the technical spike and assign it accordingly.
 - Duration will be dependent on the subject matter and Growth's familiarity with the topic.
- Review the contents of the technical spike issue body.
 - Add any comments to clarify outstanding questions with the responsible PM.
 - Identify potential needs for cross-team collaboration, subject matter expertise, or other important inputs for the

technical spike.

- Add with your proposed approach and note any desired PM checkpoints.

During the technical spike investigation period

- Document your progress with detailed notes to support our async collaboration model and maintain an inclusive environment.
- Share any discoveries that might impact the spike's duration or scope with the responsible PM.
- If approaching the due date and needing more time, proactively discuss next steps with the responsible PM.

Outputs

As the technical spike work comes to a completion, complete the following steps to close out the spike process:

- Provide a summary comment with detailed learnings of your investigation and recommended path(s) for the solution in the spike issue.
Include an outline of the recommended next step issues and/or epics to be created for the next phase of work.
- Tag in the responsible PM, EM, and Growth Engineering team (@gitlab-org/growth/engineering team) for review.
- Confirm with the responsible PM that the spike requirements have been met within the allotted duration time.

Based on your findings, proceed with one of these three outcomes:

1. For technically infeasible solutions:

Close the issue with a clear explanation of the technical limitations discovered.

2. For single-issue implementations:

Refine the technical details in the current issue and move it back into ~workflow::solution validation for PM guidance.

3. For multi-issue implementations:

Promote the spike to an epic and create the necessary implementation issues to break down the work.

As the engineer who performed the spike, you are responsible for creating these implementation issues.

For outcomes 2 and 3, ensure technical details are sufficiently documented to support implementation once it re-enters later workflow stages.

UX

Info on the Growth UX team and how they work can be found on the Product/Growth page.

How We Use Issues

To help our team be efficient, we explicitly define how our team uses issues.

Issue Creation

We aim to create issues in the same project as where the future merge request will live. For example, if an experiment is being run in the GitLab CustomersDot, both the issue and MR should be created in the CustomersDot project.

We emphasize creating the issue in the right project to avoid having to close and move issues later in the development process. If the location of the future merge request cannot be determined, we will create the issue in our catch-all growth team-tasks project.

We use issue templates for common tasks.

Iteration

To support Iteration Growth engineering:

1. Separates refactoring from feature MVCs. When refactoring is raised in review, the preference is to resolve in a follow up issue.

1. Addresses technical debt and follow up issues through prioritization and discussion by relevant stakeholders.

1. Issues with a weight of 5 and higher should be reassigned to the Product Manager to make sure they can be split into smaller MVCs.

When this is not possible, the Product Manager will create a spike or research issue so that engineering can break it down and close the original.

1. We make use of research issues and engineering spikes in the prior milestone, so estimations are more accurate.

Status Updates

The DRI (assignee) of an issue is encouraged to add an async issue update, particularly for issues labeled ~Deliverable or ~Stretch.

These updates can be added anytime and are useful to highlight when an issue is on track, blocked, or may need to be re-prioritized.

This helps us maintain transparency and our bias towards asynchronous communication.

markdown

Async issue update

YYYY-MM-DD Update

Please provide a quick summary of the current status (one sentence)

When do you predict this feature to be ready for maintainer review?

Are there any opportunities to further break the issue or merge request into smaller pieces (if applicable)?

Were expectations met from a previous update? If not, please explain why

Experimentation

The Growth groups regularly run experiments to test product hypothesis.

- Read about the experimentation process.
- View the developer documentation
- View slides for the Experimentation Workshop

GitLab team members are welcome to attend the Growth Stage Engineering Weekly meetings to connect with growth team members and find out more about running experiments at GitLab.

Growth Metrics

The Growth stage tracks number of experiments deployed as a development metric. This is not an individual or team performance indicator. This allows teams to track their technical debt.

New experiments added to the codebase using experiment feature flags are tracked until removal. Current experiments shows the total number of experiments in the codebase (pending, active, or concluded and ready to be removed).

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Experimentscadenceandcurrentexperimentsdeployed/Experimentscadenceandcurrentexperimentsdeployed" >}}
{{< /tableau >}}
```

Growth Section Metrics

Complete list of Growth Section engineering metrics.

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "SECTIONLABEL"="growth" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "SECTIONLABEL"="growth" >}}
{{< /tableau >}}
```

Team Days

On occasion we hold virtual team days or meetings to take a break and participate in fun, social activities with our Growth counterparts.

- FY23-Q2 2022

- December 2021
- April 2021
- September 2020
- May 2020

Common Links

- Growth stage
- Growth workflow board
- sgrowth in Slack (GitLab internal)
- Growth opportunities
- Growth meetings and agendas (GitLab internal)
- GitLab values

SECTION: engineering/development/growth/acquisition.md

Vision

The Acquisition Group is part of the Growth Stage]. We work on [connecting our users with our product value.

- I have a question. Who do I ask?

Questions should start by @ mentioning the Product Manager for the Acquisition group or creating a new issue in the Growth Product [Acquisition issues] list.

Mission

The Growth:Acquisition Group works on product priorities and growth deliverables with our Growth product managers and contributes to Growth initiatives including the Registration Flow and Continuous onboarding.

Team Members

The following people are permanent members of the Growth:Acquisition team:

```
{{< team-by-manager-role role="Engineering Manager(.)Growth and Govern:Threat Insights" team="Acquisition" >}}
```

Stable Counterparts

We work directly with the following team members in the Growth:Acquisition group:

```
{{< engineering/stable-counterparts role="Growth(.)|Growth:Acquisition" manager-role="Engineering Manager(.)Growth and Govern:Threat Insights" other-manager-roles="Director(.)Growth(.)|Engineering Manager(.)Govern, and Growth|Sales|Activation" >}}
```

How We Work

- We're data savvy
- In accordance with our GitLab values
- Transparently: nearly everything is public
- We get a chance to work on the things we want to work on
- Everyone can contribute; no silos

Prioritization

Prioritization is a collaboration between Product, UX, Engineering, and Quality.

- We use the ICE framework for experiments
- We use Priority and Severity labels for bugs

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="acquisition" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}  
  {{< tableau/filters "GROUPLABEL"="acquisition" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}  
  {{< tableau/filters "GROUPNAME"="acquisition" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="acquisition" >}}  
{{< /tableau >}}
```

Workflow

- We use the Product Development workflow when working on issues and merge requests across multiple projects.
- We use a shared Kanban workflow board for both groups - Acquisition and Activation for work in the build phase of the product development flow.
- We use refinement threads on issues to discuss our approach and estimate weight of upcoming work.

Common Links

- Growth Stage
- Acquisition issues
- gacquisition in Slack (GitLab internal)

SECTION: engineering/development/growth/activation.md

Vision

The Activation Group is part of the Growth Stage]. We work on [connecting our users with our product value.

- I have a question. Who do I ask?

Questions should start by @ mentioning the Product Manager for the Activation group or creating a new issue in the Growth Product [Activation issues] list.

Mission

The Growth:Activation Group works on product priorities with our Growth product managers and contributes to Growth initiatives including the Registration Flow and Continuous onboarding.

Team Members

The following people are permanent members of the Growth:Activation team:

```
{{< team-by-manager-role role="Engineering Manager(.)Growth and Govern:Threat Insights"
team="Activation" >}}
```

Stable Counterparts

We work directly with the following team members in the Growth:Activation group:

```
{{< engineering/stable-counterparts role="Growth(.)|Growth:Activation" manager-
role="Engineering Manager(.)Growth and Govern:Threat Insights" other-manager-
roles="Director(.)Growth(.)|Engineering Manager(.)Govern, and Growth|Sales|Activation" >}}
```

How We Work

- We're data savvy
- In accordance with our GitLab values
- Transparently: nearly everything is public
- We get a chance to work on the things we want to work on
- Everyone can contribute; no silos

Prioritization

Prioritization is a collaboration between Product, UX, and Engineering.

- We use the ICE framework for experiments.
- We use Priority
and Severity labels for bugs.

```
{{< tableau height="600px" toolbar="hidden" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/TopEngineeringMetrics/TopEngineeringMetricsDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="activation" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/MergeRequestMetrics/OverallMRsbyType1" >}}
  {{< tableau/filters "GROUPLABEL"="activation" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/Flakytestissues/FlakyTestIssues" >}}
  {{< tableau/filters "GROUPNAME"="activation" >}}
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://us-west-2b.online.tableau.com/t/gitlabpublic/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="activation" >}}
{{< /tableau >}}
```

Workflow

We use the Product Development workflow when working on issues and merge requests across multiple projects.

We use the Activation development workflow board for work in the build phase of the product development flow.

Common Links

- Growth Stage
- Activation issues
- gactivation in Slack (GitLab internal)

SECTION: engineering/development/growth/experimentation.md

Experimentation

This page covers the Growth engineering process for running experiments. See also:

How Growth Launches Experiments (Product)
Experiment Guide (GitLab Developer Documentation)

Experimentation Design & Analysis (Product Analysis)

Running Experiments

We follow a four step process for running experiments as outlined by Andrew Chen's How to build a growth team.

1. Form Hypotheses: Define ideas our team wants to test.
2. Prioritize Ideas: Decide which ideas to test first.
3. Implement Experiments: Do the Product, Design, Engineering, Data, and Marketing work to execute the experiment.
4. Analyze Results: Dive into the results data and prove or disprove our hypotheses.

Each week, we provide progress updates and talk about our learnings in our Growth Weekly Meeting.

The duration of each experiment will vary depending on how long it takes for experiment results to reach statistical significance. Due to the varying duration, there will be some weeks when we have several experiments running concurrently in parallel.

Experimentation Process

Overview

1. Create an Experiment idea issue to define the scope of the experiment (label added: ~"experiment idea", optionally add: ~"growth experiment")

Anybody who has an idea of something to try as an experiment can create this issue. They provide information about which portion of the product they'd like to experiment with, their experimentation idea (i.e. their hypothesis), and what a successful outcome might look like.

At this extremely early stage, the most critical thing is that we get the idea jotted down with enough context and detail that it is understood by others on the team and can be carried forward by the Project Manager.

1. Project Managers (PM) review the Experiment backlog regularly to ensure they meet the necessary criteria of a well-defined experiment

experiments that meet our Experiment Definition Standards and are deemed relevant are initially prioritized by adding the appropriate milestone list (%Awaiting further demand, %Backlog, %Next 1-3 releases, or a specific milestone)

experiments that do not meet our Experiment Definition Standards or are deemed no longer relevant are closed

1. PM follows the process outlined in Experiment Setup to create an epic and related issues

1. PM, UX, and Engineering follow the product development flow and create workflow:: issues linked to the epic to track the work required to complete the experiment

1. Additionally, the PM works with the Data Team to define what data will be needed in order to fulfill the defined success metric

Note: In order for the Data Team to begin ingesting data from new tables or columns, those tables or columns must first exist in the production database.

1. Engineering teams deliver changes in line with our product development flow, being mindful

of changes affecting other GitLab teams.

Our processes, planning, and current priorities are transparent to other groups
We try to avoid running experiments on features that are currently or soon to be under development by other groups

We choose reviewers and domain experts from relevant groups where possible to ensure different groups are familiar with the changes we are introducing

When our product managers make the call, we initiate experiment and feature flag cleanup, either adding the experiment feature to the product or reverting

1. Engineering & PM collaborate on getting the experiment activated according to the defined rollout plan

PM or Engineering update the experiment rollout issue, setting the experiment scoped label to ~"experiment::active" when the experiment is live

PM or Engineering notify interested parties (their team, data team, support, CSMs, etc.) by at-mentioning them in a comment on the experiment rollout issue, and on the production Slack channel

1. PM monitors the experiment via data sent to Sisense, adjusting the rollout strategy in the experiment rollout issue as necessary

1. PM compares recorded data to the experiment's criteria for measuring success
if experiment (variant) is successful, PM/Engineering create an Experiment Cleanup Issue to fully integrate the successful flow into the product

if experiment (all variants) are unsuccessful, PM/Engineering create an Experiment Cleanup Issue to remove the experimentation code, reverting back to the "control" flow

1. Once the experiment cleanup issue is resolved, the experiment rollout issue and Experiment Epic are closed and the experimentation process is complete

See also the Growth RADICIE and DRIs for determining DRIs at each stage.

Experiment Issue Boards

A backlog of experiments are tracked on the Experiment backlog board.

To track the status of an Experiment, Experiment tracking issues using the ~"experiment-rollout" and scoped experiment:: labels are tracked on experiment rollout boards across the following groups:

| gitlab-org | gitlab-com | all groups |
| ----- | ----- | ----- | ----- |
| Experiment rollout | Experiment rollout | Issues List |

Experiment Setup

Once the experiment hypothesis & measurement criteria are well-defined, an Experiment Epic is created at the top-level group most appropriate for the experimentation work (usually the gitlab-org group)

The epic becomes the single source of truth (SSoT) for the experiment

The original Experiment Definition Issue is closed and gets attached to the epic

The final definition, hypothesis, & measurement criteria from the Experiment Definition Issue are copied into the new epic's description

New issues related to the experiment (such as an experiment rollout issue, an experiment cleanup issue, a UX spec issue, or an Engineering work issue) are attached to the epic

Experiment Definition Issue

This issue acts as the starting point for defining an experiment, including an overview of the experiment, the hypothesis, and some idea of how success will be measured. The [Experiment idea] issue template can be used for this (label added: ~"experiment idea", optionally add: ~"growth experiment").

Experiment Definition Standards

1. The experiment can be qualitatively measured (i.e. there are no other factors that could give us false data)
1. The experiment is as atomic as possible (i.e. it is easiest to measure one, specific, individual change rather than a series or group of changes)
1. The experiment has clearly defined and reliably measurable success metrics
 - There is consensus around the chosen success metric
 - The success metric is clearly tied to a piece of data which will be collected & measured during the active phase of the experiment (experiment rollout issue)
1. The experiment includes an estimated time to significance: how long the experiment would need to run given the population size and expected conversion rate
 - Can use handy tools, like the A/B-Test Calculator (external), to help calculate this estimate

Experiment Epic

This epic acts as the single source of truth (SSoT) for the experiment once an experiment has been properly defined according to our Experiment Definition Standards and is deemed worthwhile to run. Once an Experiment Definition Issue is added to this epic, we fill out further details such as the expected rollout plan. We also assign the experiment to a milestone and follow the product development flow for UX & Engineering work. As the experiment design and rollout progresses, this epic or parent issue should contain details or links to further information about experiment including the tracking events and data points used to determine if the experiment is a success as well as links to relevant metrics-reporting dashboards (such as Sisense).

Experiment Rollout Issue

This issue is used to track the experiment progress once deployed. It is similar to a Feature Flag Roll Out issue with an additional experiment:: scoped label to track the status of the experiment. The [Experiment Tracking] issue template includes an overview of the experiment, who to contact, rollout steps, and a link to the overall experiment issue or epic.

The experiment:: scoped labels are:

- ~"experiment::pending" - The experiment is waiting to be deployed
- ~"experiment::active" - The experiment is active (live)
- ~"experiment::blocked" - The experiment is currently blocked
- ~"experiment::validated" - The experiment has been validated (the success criteria was clearly

met)

~"experiment::invalidated" - The experiment has been invalidated (the success criteria was clearly unmet)

~"experiment::inconclusive" - The experiment was inconclusive (the success criteria was not clearly met nor clearly unmet)

Experiment Cleanup Issue

This issue is used to clean up an experiment after an experiment has been completed. It is created within the project where the cleanup work will be done (e.g. the gitlab-org/gitlab project).

The cleanup work may include completely removing the experiment

(in the case of ~"experiment::invalidated" and ~"experiment::inconclusive") or refactoring the experiment feature for the long run (in the case of ~"experiment::validated").

The cleanup issue should be linked to the experiment rollout issue as a reference to ensure the experiment is concluded prior to cleanup.

The Experiment Successful Cleanup issue template can be used for the gitlab-org/gitlab project.

Experiment issue templates

GitLab gitlab-org/gitlab project

- Experiment idea issue template

- Experiment implementation issue template for Engineering

- Experiment rollout

- Experiment successful cleanup issue template for converting a successful experiment into a feature

Growth team-tasks project

- Milestone planning issue template for Growth groups

- Experiment idea issue template for Growth groups

- Experiment rollout issue template for Growth groups

Minimum Viable Experiment (MVE)

Experimentation, like everything at GitLab, should be approached with the GitLab CREDIT values in mind, specifically the values of Iteration, Efficiency, and Results.

The larger an experiment is, the longer it takes to craft a design, implement code changes, review code changes, define and collect necessary data, organize data into meaningful tables, graphs, and dashboards, and so on.

As we build and improve our experimentation platform and increase our ability to quickly create and run experiments, we should expect a large percentage of all experiments to fail at proving their hypotheses.

Given these invalidated or inconclusive experiments will be rolled back there is an advantage in ensuring experiments are as small and iterative as possible.

With this in mind, there are advantages to considering developing a Minimum Viable Experiment (MVE).

Much like the concept of Minimal Valuable Change (MVC), the goal of an MVE is to look for the smallest hypothesis to test, the simplest design for testing that hypothesis, the quickest

implementation of the design, the least amount of data to be collected in order to verify the hypothesis, and so on.

So, what might an MVE look like in practice?

Matej Latin shares an example of a so-called "painted door" test in his blog post, "Small experiments, significant results and learnings".

A simple example of a "painted door" test might be a call-to-action (CTA) button that doesn't really go anywhere - maybe it brings up a simple modal which says "Oops! That feature isn't ready yet," or maybe it takes the user to an existing page in our documentation.

Because the design of this type of MVE is intentionally simple, it is easier and faster to develop, deploy, and start gathering data.

Because the design of this type of MVE is intentionally simple, it is easier and faster to develop and deploy. With a small amount of instrumentation, we can use it as an opportunity to measure initial engagement with that button.

Who clicks on the CTA?

How often is it getting clicked?

This can be a fairly low cost way to inform next steps, for example rolling back, developing a larger experiment, or implementing a feature as a follow-up.

Keep in mind that "painted door" tests are not always the best first approach.

The main idea is to strive for an iterative approach to experimentation.

Ask yourself, "Is there a simpler version of this experiment which is worth deploying and which still gives us enough data to know how to proceed?"

Experiment Status

For real time experiment rollout status GitLab team members can view the experiments API (docs) (a JSON viewer for your browser is recommended).

The "currentstatus" will be on, off, or conditional. If conditional, there will be either a percentageoftime or percentageofactors. Refer to the note on feature flags in the experiment guide.

There are dashboards in Sisense to indicate whether the experiment flag still exists, but not the current status. These are also only available to GitLab team members.

The experiment rollout board lists rollout issues linked to experiment feature flags used in development and on SaaS.

SECTION: engineering/development/growth/fast-boot.md

The Acquisition, Expansion, Conversion and Retention groups took part in a Fast Boot in September 2019. The planning issue contains the proposal for Fast Boot, and this page documents our discussions and outcomes.

Day 1

Link to Day 1 Agenda Document

Key Outcomes

How we operate as a team

Section Level

Monthly Product/Growth metrics review with E-Group (VP Product leads, with support from Growth Director)

Every 6 weeks Growth Group Conversation (PM + Eng Director lead)

Weekly engineering meeting

Weekly ideation & prioritization meeting

On Monday, check in on experiments to identify which can be concluded, and to collect data for review meeting on Tuesdays

Group Level

Groups will organize/meet as needed

How do we run Growth experiments?

Development board labels/process - continue on & follow handbook

A/B testing process, artifacts, expectations

Each experiment will set confidence level - 95%-99%

Test summary written by data analyst in the issue (add a "a/b test complete" label for searching later)

Expectations for working in another teams' code base

Emphasize that the point is to gather data, not ship production code

Challenge: review process too slow. Reviewers vs Maintainers. Only a small number of maintainers.

Goal: one deliverable picked up per week

Feature Ownership

Fulfillment owns underlying billing, licensing, and transactional system, as well as current portal user experience.

Acquisition owns the New Signup ARR growth KPI + the new signup UX

Conversion owns the New Free to Paid Trial ARR growth KPI + the trial UX

Expansion owns the Current customer seat expansion + current customer upgrade ARR growth KPI + the upgrade flow & true-up flow

Retention owns the Gross Retention KPI + renewal flow

Growth PM's will own issues for key flows; engineering leaders will help assign work

Long Term: the vision is for the fulfillment team to own the billing and licensing platform, with a set of API's and SLA's, so the growth teams can build and curate user experiences on top

of them to drive great customer experiences (and resulting KPI's).

Deep Dives

Teardown/journey map existing flow. Leverage demos, videos, screenshots. Ask "experts" to join remotely to demo and share their pain points/opportunities. Session to be led by the product manager.

Acquisition Overview

Discussion led by Jensen, PM for Acquisition, topics covered include:

- KPIs and mission for the Acquisition team
- Initial areas of focus
- Teardown of the existing experience
- Teardown of the competitors experience
- Identifying opportunities for improvement

Video

Conversion Overview

Discussion led by Sam Awezec, PM for Conversion, topics covered include:

- KPIs and mission for the conversion team
- Initial areas of focus (Initial product activation, Ah-ha moments with paid features and/or limits, Trial experience and overall value)
- Teardown of the existing experience
- Identifying opportunities for improvement

Video

Expansion Overview

Discussion led by Tim Hey, PM for Expansion. Topics covered:

- Mission
- KPIs & Supporting Performance Indicators
- Customer facing opportunities
 - User Orientation - Users don't know where to start
 - Increase platform confidence and trust - I love my tools and am afraid to switch
- Internal opportunities
 - Self-Managed usage for upsell process and user workflow
 - Self-Managed True-up process and user workflow

Video

Slides

Retention Overview

Discussion led by Mike Karampalas, PM for Retention. Topics covered:

- KPIs for retention
- Near-term and longer-term focus for the Retention team
- Teardown of the renewal flow(s)
- Opportunities for improvement

Video

Slides

Day 2

[Link to Day 2 Agenda Document](#)

Data Deep Dive

Discussion led by Eli Kastelein, Growth Data Analyst. Topics covered:

- GitLab.com data vs Self-Managed data
- Overview of data sources relevant to the growth team
- Vision for how we can use data to serve the growth team
- Event-based reporting versus database reporting
- What's currently blocking us from collecting the data we need to be successful

Video

Slides

State of Product Intelligence

During this time we walked through Product Intelligence as a team and reviewed where we were headed. The main item we covered was the Vision Page in the handbook and how we need to clearly communicate what product analytics is and where to find more information.

Topics covered (which will also be added to the handbook)

- Vision
- SMAU -- what it is and how it's managed
- Dashboards -- what they are, self-serve and how to request one
- Data fields available for tracking and what to do if you need a new one
- Tracking and instrumentation -- how to's for GitLab.com and Self-Managed
- Technologies we use

MR under review

A/B Testing Requirements

Issue: Implement an A/B testing solution that can measure the impact of Growth experiments

Key outcome is to work with the Release team to improve our existing Feature Flags capability in GitLab in order to successfully A/B test Growth experiments.

Right now in order to run an experiment, we need to do additional work to ensure we can measure the results. This work will be bespoke for each experiment while we improve our capability.

We can investigate third party solutions if needed.

Days 3-5

At this time the individual Growth teams split into their groups to create and work on issues in their respective areas.

Fast Boot Outcomes

Implemented new team names, KPI's and objectives

Rename Growth Groups

Updating growth KPIs on KPI index page

Streamlined team processes

Product/Engineering meeting cadences and deliverables established

Product/Engineering workflow established

A/B Testing discussion progress

Create and deliver issues

Acquisition

Test simplified user registration page + Design progress

Update .com paid signup process

Update trial signup process

Update Self-Managed paid signup process

Expansion

License Utilization by Host - Epic

Add seat differential to host page on version.gitlab.com

Add column to the host page of version.gitlab on the host page to show last check date

Add a column to version.gitlab on the host page to show if usage ping is available

From license.gitlab.com link to the License search results page on the Hosts page of version.gitlab

Search by license on version.gitlab

Add column sort functionality to the host page on version.gitlab.com

Conversion

Adjust in-app pricing page links

For free users of GitLab.com add trial options for bronze and silver

- Add project import to the existing welcome to GitLab page
- Create end to end testing for trials
- Remove duplicate trial banner CTA from group billing page
- Update group creation URL so it is always unique and does not generate an error
- Group billing page should contain options to buy now and start a trial
- In group trial add buy now button
- Change "welcome to GitLab" message
- Paid feature upgrade point - feature weights

Retention

- CustomersDot: change copy on renew page
- Update the automated Zuora renewals email
- Link to CustomersDot in renewal banner
- Make Renew or Add Users the primary CTA on Manage Purchases instead of "Add new subscription"
- Add link to customers.gitlab.com from gitlab.com and about.gitlab.com
- Charge for actual users when Auto Renew is enabled for .com customers
- Enable Deep Linking to the Renewal Flow
- Fix Renewal Emails to Not Come from paul@gitlab.com
- Consolidate the emails generated by a renewal
- WIP: Make Auto Renew the Default

SECTION: engineering/development/onboarding/_index.md

SECTION: engineering/development/onboarding/manager.md

Training

Development onboarding training

Much of the training Engineering Managers (EMs) should consider is already part of our onboarding. As time progresses, we may add additional things, so a periodic review of development department, people manager, and becoming a GitLab manager onboarding activities for new actions is valuable.

Diversity, Inclusion, and Belonging Training

Diversity, Inclusion & Belonging is one of GitLab's 6 values. It's what we believe distinguishes us from other companies. Engineering Managers should take DIB Training which reviews key sections of our handbook and a learning path. We all need to work on making sure we are inclusive as we work together to build some really great things.

Iteration Training

Besides normal onboarding training with various tools and processes. Engineering Managers should also take the Iteration Training. Engineering Managers can start the process of training by following the steps in this template. Iteration is a important value of GitLab and not easily learned. Working with other Engineering Managers and Product Managers during this exercise is encouraged to further enhance the experience.

SECTION: engineering/development/processes/_index.md

SECTION: engineering/development/processes/follow-the-sun-coverage.md

Follow the Sun Coverage, also known as Follow-the-Sun Model, is a strategy commonly used by global organizations or teams that operate in different time zones. It aims to ensure continuous support and productivity by leveraging time zone differences.

In the Follow the Sun Coverage approach, teams are organized across multiple regions or locations around the world, typically spanning different time zones. As one team's workday comes to an end, another team in a different region takes over seamlessly, ensuring round-the-clock coverage and uninterrupted progress on critical deliverables.

The main idea behind Follow the Sun Coverage is to maximize efficiency and reduce turnaround time by leveraging the availability of team members across different time zones. This allows for work to be passed on from one team to another, ensuring that projects or tasks keep moving forward even when individuals in a particular region are not actively working.

How GitLab uses Follow the Sun Coverage

- Support and Issue Management: GitLab may organize its support teams across different regions to provide continuous assistance to users, customers, and clients. By following the sun, they can ensure that support coverage is available during business hours across multiple time zones, minimizing response times and providing a seamless experience.
- Development and Collaboration: GitLab's development teams, spread across different regions, can leverage Follow the Sun Coverage to ensure continuous progress on projects. This approach enables developers to seamlessly hand off work to team members in other time zones, ensuring that development efforts proceed without interruption and taking advantage of the diverse expertise and perspectives available.
- Incident Response: In the event of critical incidents or emergencies, GitLab can employ Follow the Sun Coverage to have incident response teams available around the clock. By having teams distributed across different time zones, they can respond promptly to incidents and minimize the impact on customers and services.
- Project Management: GitLab may utilize Follow the Sun Coverage to enable effective project management across global teams. With team members working in different time zones, project updates, coordination, and decision-making can happen continuously, ensuring smooth progress and reducing bottlenecks caused by time zone differences.

Follow-the-Sun Pairing

In time-sensitive situations, we employ a "Follow-the-Sun Pairing" strategy to expedite the delivery of critical deliverables across EMEA, APAC, and AMER regions.

This approach involves forming a group of multiple individuals, each representing one of the regions, to collaboratively author the deliverable. By leveraging time zone variances, the group ensures continuous progress on the deliverable, with one team member seamlessly taking over as another's workday ends. This promotes uninterrupted work throughout the day, maximizing efficiency.

The group members may come from different stages or sub-departments, fostering cross-functional collaboration and drawing upon diverse expertise and perspectives. This strategy reflects our commitment to global collaboration and effective time management, enabling us to accelerate deliverables while ensuring comprehensive support across all regions.

This way of pairing is less efficient but faster than if one person worked on the deliverable. The hand off must be context-rich. The first pair has to describe all design decisions in detail so the second pair knows which choices were deliberate. This will prevent rework by the second pair. This differs from normal code review, where less context means that the reviewer asks a question (which is significantly less overhead than re-implementing part of the feature).

To ensure effective hand offs and coordination between team members, we have implemented an organizational structure and designated a "Follow-the-Sun DRI" (Directly Responsible Individual) for overseeing the process. The Follow-the-Sun DRI is responsible for coordinating the hand off between regions and facilitating communication among team members.

Note: It's important to assess the suitability of the "follow-the-sun" approach. This approach is best used in the event of escalations or high priority tasks that come up. Examples of this may include (but are not limited to):

- Critical customer fixes
- High severity security issues or
- Corrective actions related to a high severity incident

With this level of urgency to be confirmed by the group's leadership (e.g. Engineering Manager and Product Manager).

Futher reading and examples

- Verify - Managing Urgent Priorities
- MVC - GitLab WebIDE support - Code Suggestions

SECTION: engineering/development/processes/hiring-contractors.md

GitLab normally prefers to hire full-time permanent engineers for its development teams using entities and PEOs. However, there are times when teams will have a need to hire temporary

contingent workers to help with execution.

These are a few examples of situations where a contingent workers may be required:

- A team member is on long-term leave and the team needs to back-fill that position for a period of time.
- There is a one-off project that requires timely execution and other team members are busy with higher-level priorities.
- The team may lack very unique expertise or technical specialization and a consultant is needed to help.
- Provide financial support for contributions to an open-source project by an external party.

Whatever the reason, there are some unique challenges and processes related to hiring contracontingent workers, which makes it different from a regular hire.

In the next sections, you can find general information about the process. However, you should consult with your sub-department lead for any variations to this process. You should also read the GitLab [Contingent Worker Policy] (</handbook/finance/procurement/contingent-worker-policy/>) as a pre-requisite.

Planning and budget approval

Like with full-time hires, contingent workers will require planning ahead of time for the budget necessary to cover the time expenses according to your needs.

For this, you need to discuss with your sub-department leader the need and justification for a contingent workers. Plus, you need to work the numbers in terms of the level of experience required and the length of the contract. You also need to determine if the contingent workers will be working full-time hours or part-time.

When considering the budget, keep in mind that contingent workers work for salary only. There are no benefits for them. You should also consider if you will hire directly or if you will engage with a hiring agency to find them for you. The fees might be very different in these two scenarios. When working with a recruitment or team augmentation agency you will need to account for hiring fees or other administrative costs charged by the agency.

It is advisable to put together a concise proposal where you outline the objectives you are looking to meet by hiring a contingent workers, justifying clearly the reasons you can't use a full-time team member hire to meet the needs. Also include budget and timeline information in the proposal. Here is one example of an issue that outlines the need and seeks budget approval.

Once you agree on the budget with your manager and get approval at the department head level, you can move on to the next phases of the hiring process.

Candidate Sourcing

The first thing to be aware of about sourcing contingent workers candidates is that they are hired as professional services vendors through a procurement process. This means that as an engineering manager you get little to no support from talent acquisition recruiters at GitLab.

You should consider your strategy. You may reach out to individuals and hire them directly by using your professional networks or tools such as LinkedIn Recruiter. You can also engage with a recruitment agency or team augmentation agency for this purpose.

If you choose to hire them directly, you will have less administrative work to get the agency vetted and approved by the procurement department. However, you will have to invest more effort in the sourcing work yourself.

One thing you can do is to ask GitLab recruiters to help you and be on the lookout for past candidates in Greenhouse that were not selected for similar jobs in your group or other groups, but who were still very strong candidates. They may make a great contingent workers for your needs and you can benefit from the context and the interview notes that will be available for those candidates as a starting point.

Going with a staff augmentation or recruitment agency can save you time on sourcing efforts, but you need to invest some additional overhead on vetting the agency itself. In this case, you may want to ask the Finance department and explore agencies already working with GitLab first. You will need to work with the selected agency to understand their policies and the way they work with companies to source and interview candidates.

In either case, you need a job description (JD). If you are hiring for a temporary backfill, you will want to have your JD be as close as possible to the job family of the role. Otherwise, be as specific about the technical and job requirements as possible.